DHSVM Spatial & Stream Network Toolkit

(that actually works), from a watershed boundary to stream networks and a complete DHSVM input set

Yiyun Song, ECS

August 17, 2026

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks

What I built this summer

- **DHSVM**, the distributed hydrology model I use for post-fire watershed work, requires a demanding input set:

  gridded terrain binaries · a **routed, ordered stream network** · **initial model states**

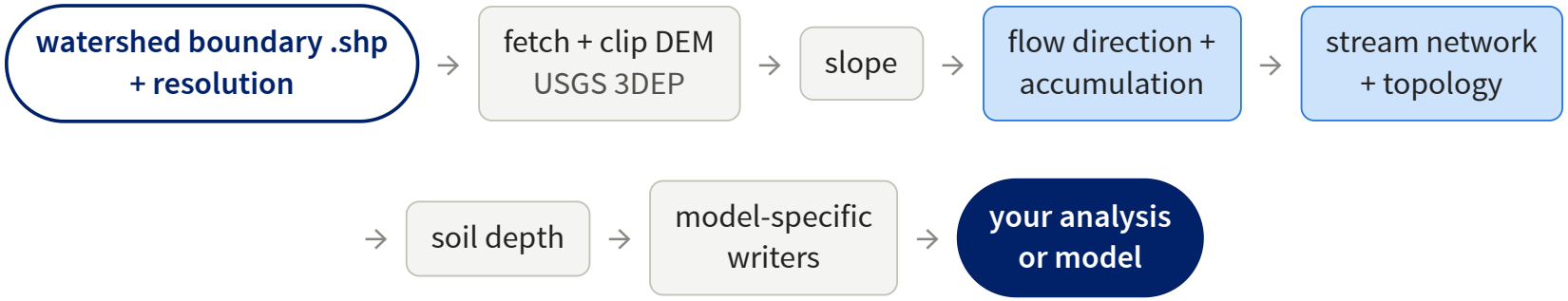
- This summer that preparation became a **toolkit:**

  one command, from a **watershed boundary shapefile + DEM** to the complete input set

- Most outputs are **not DHSVM-specific**:

  the **stream network** in particular serves many kinds of watershed analysis

What the toolkit does



General outputs (any model or analysis)

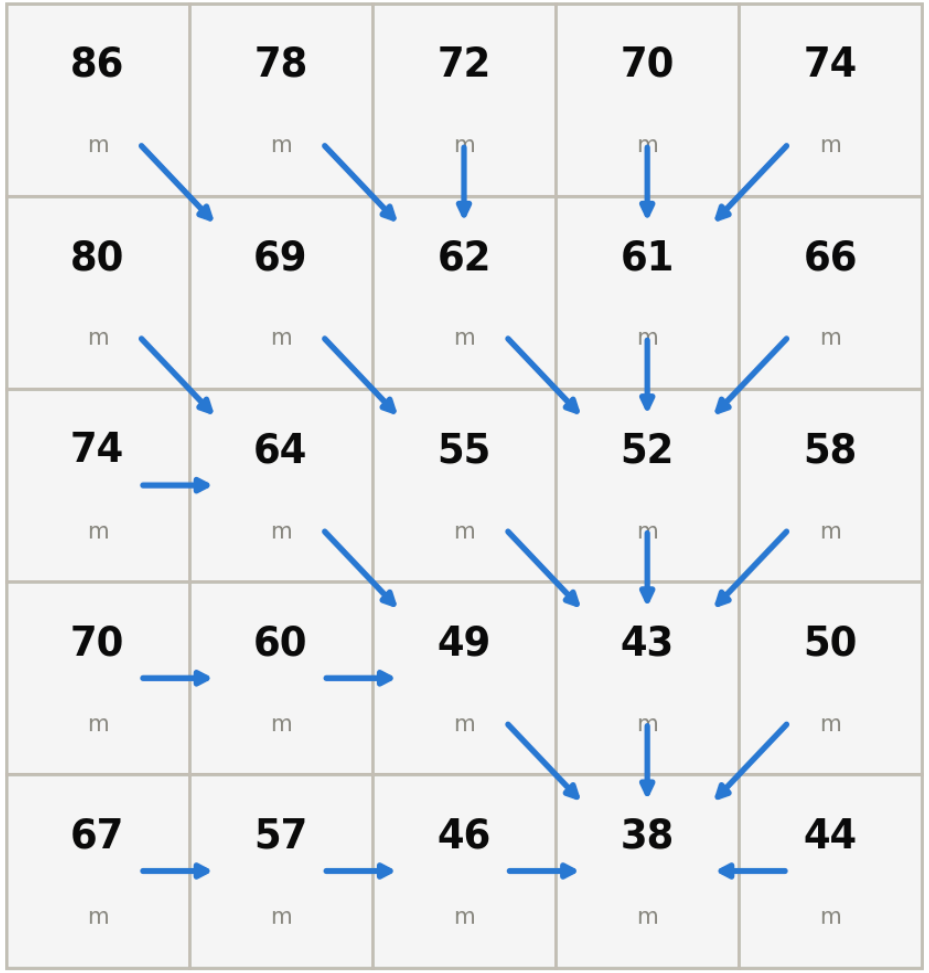
```
elev_clipped.tif      clipped DEM
slope_filled.tif      slope
flow_acc.tif          flow accumulation
flow_dir.tif          flow direction
streamfile_attr.shp   network + attributes
quicklook.png         review figure
```

DHSVM outputs (a thin final stage)

```
DHSVM_input_binaries/  dem  mask  soil
                       veg  soildepth  (.bin)
DHSVM_input_streams/   stream.class.dat
                       stream.network.dat
                       stream.map.dat
modelstate/            Interception  Snow
                       Soil  Channel
```

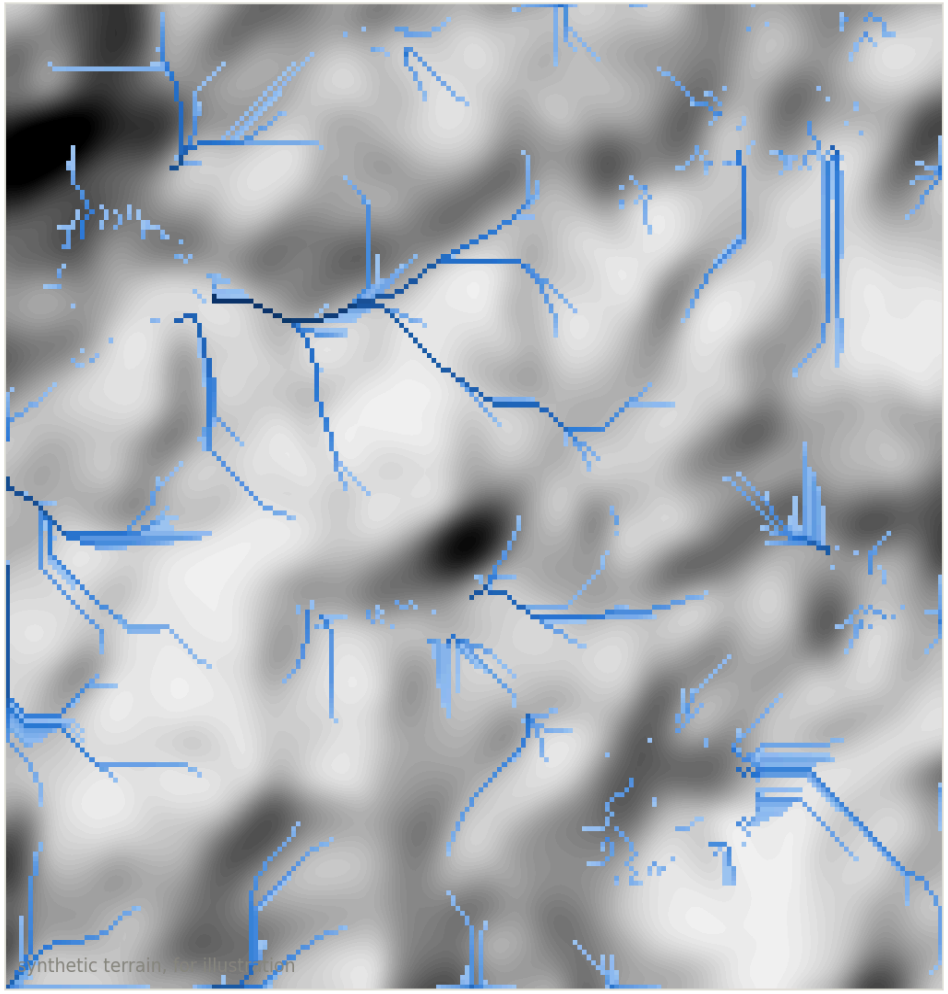
Setting up a new basin means setting a few environment variables. The pipeline runs without a GUI, so it batches on the cluster.

Water flows downhill: every cell points to one neighbor



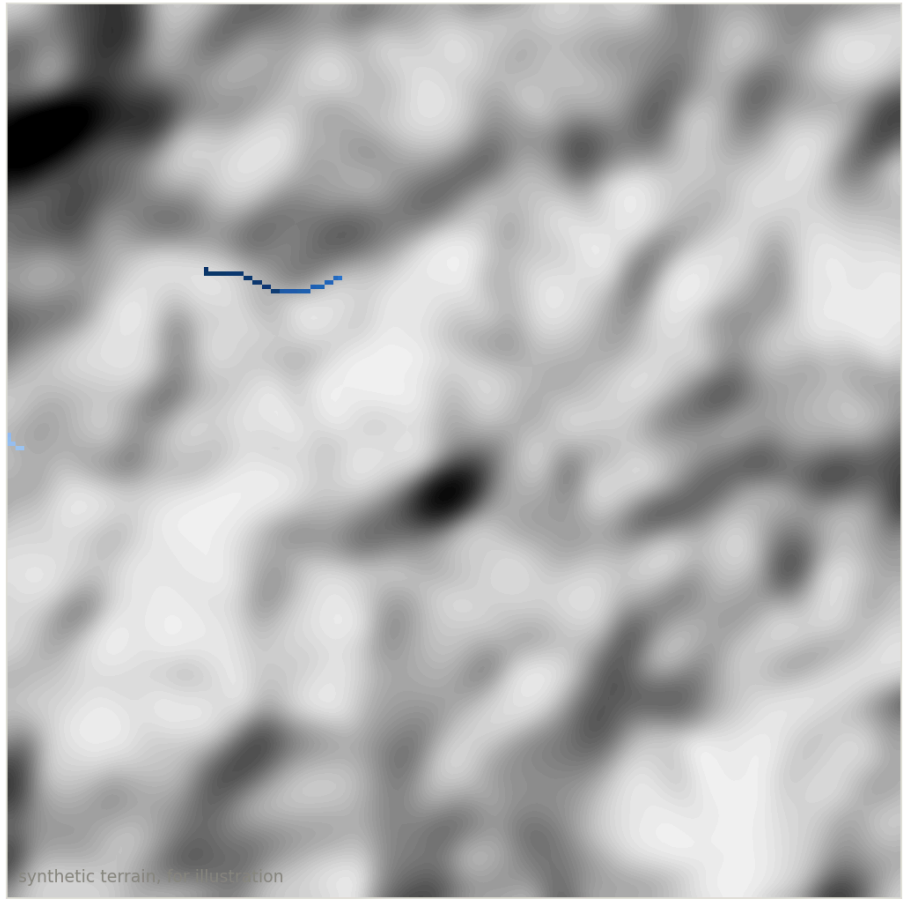
Count the drops passing through each cell

flow accumulation: the valleys emerge on their own



A stream begins where enough land drains into it

support area A_c = 2 km² drainage density 0.03 km/km²

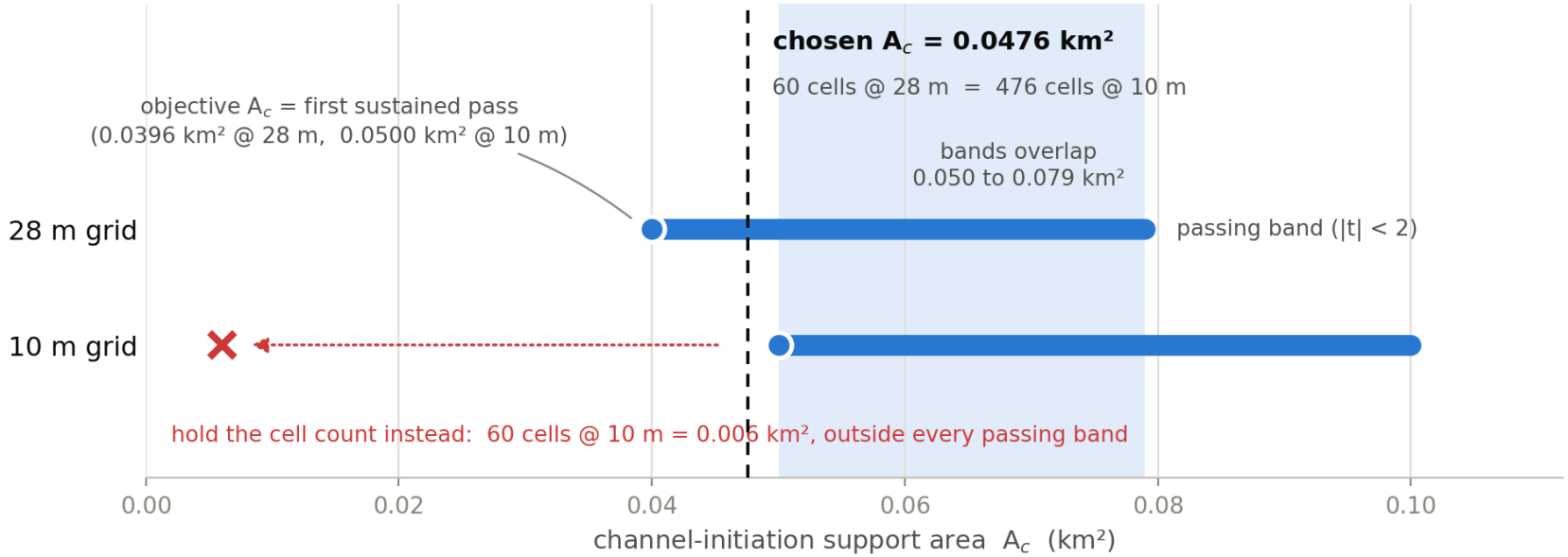


large A_c, few streams — small A_c, streams everywhere

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks

Choosing the threshold objectively

A network extracted from a DEM depends on one free parameter, the support area A_c. The toolkit selects it with the constant stream drop test: in a properly extracted network, first-order streams show the same mean elevation drop as higher-order streams. The sweep takes the first sustained band where the test passes ($|t| < 2$).

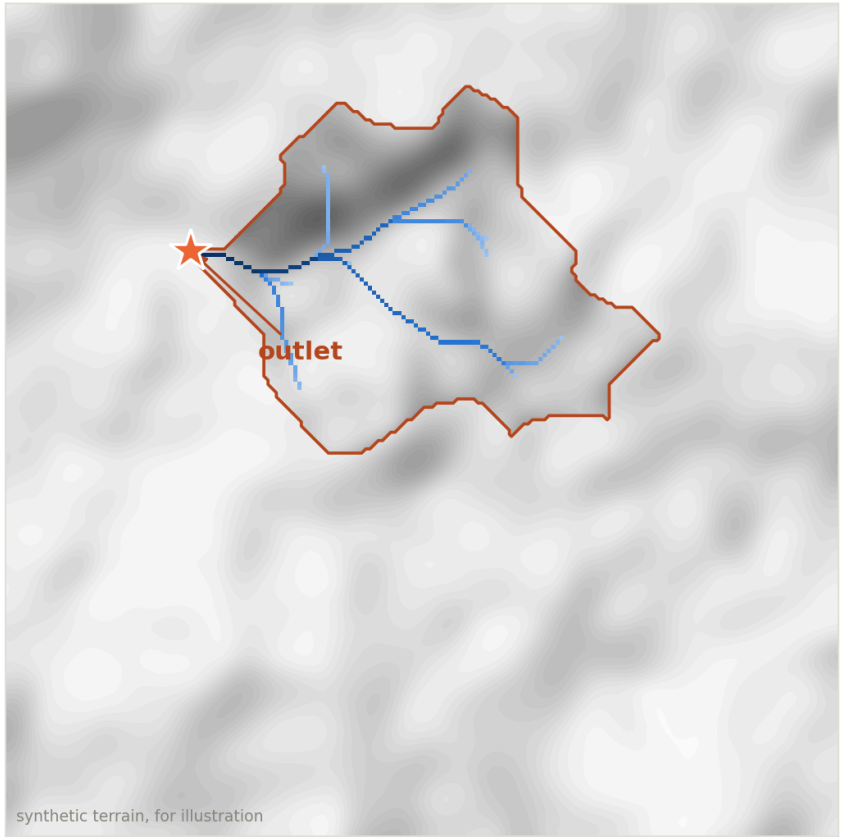


Because A_c is a physical area, the same value remains valid when the grid changes from 28 m to 10 m. Holding the cell count fixed instead would give a network about eight times too dense at 10 m.

From channel cells to a network

Then make it a network: directed, ordered, one outlet

the watershed = every cell whose water reaches this outlet

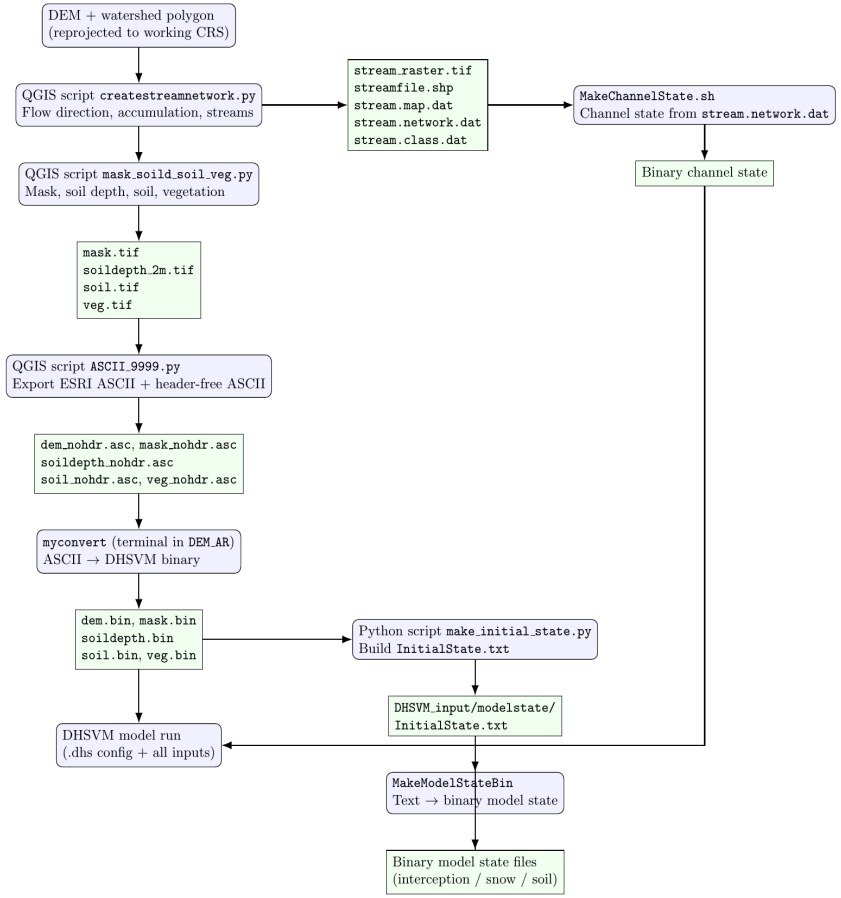


Channel cells are vectorized into segments, directed downstream, ordered from headwaters to outlet, and connected to a single outlet. Each segment carries its length, slope, drainage area, and channel class.

This topology is what routing requires, in DHSVM or any other model. It also supports transport along the network, for temperature, sediment, or nutrients, and connectivity measures for stream ecology.

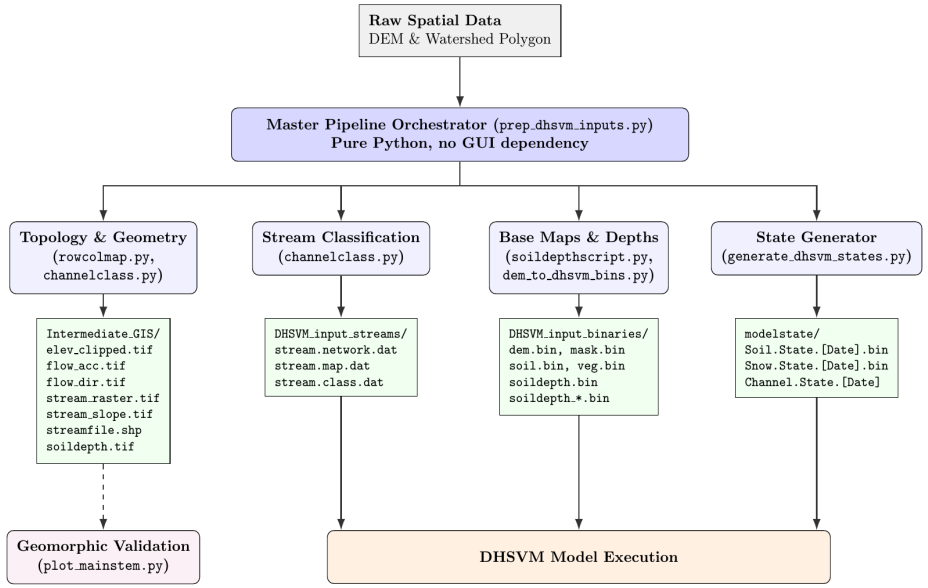
Distance ties at junctions are resolved by a fixed rule, so the same boundary gives the same network in every run.

Cleaning up the original workflow



eight steps · three format conversions · GUI required

one pipeline · Python + GRASS · headless on the cluster



Running it on the Duke Compute Cluster

```
ssh dcc                # dcc-login.oit.duke.edu
git clone https://github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks.git
source /hpc/group/abmurraylab/ys451/env.sh
   # Anaconda3, conda env dhsvm_geo, GRASS/7.6.0

export DHSVM_EPSG=32617
export DHSVM_WATERSHED=.../arwd_watershed_UTM17.shp
export DHSVM_DEM_SOURCE=.../AR_src_3dep_10m.tif
export DHSVM_OUT=/work/ys451/cct_demo/outputs_AR_10m
export DHSVM_STREAM_SOURCE_AREA_M2=22000
bash standalone_CA/pipeline/run_pipeline.sh
```

This small basin runs in a login shell; production runs go inside a Slurm allocation.

waveletswave / DHSVM_Stream_Toolkit_ThatWorks

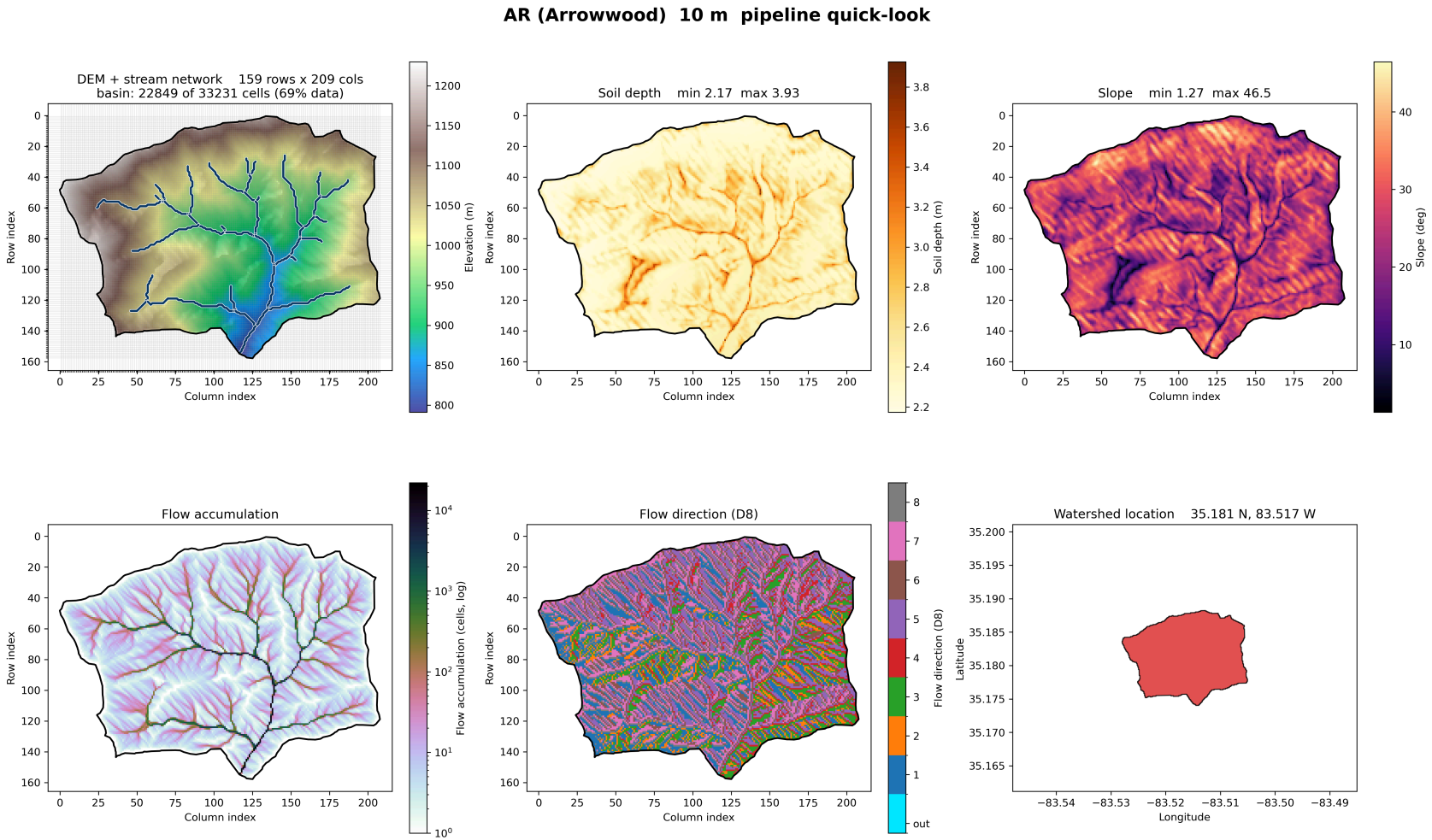
public · MIT license · guides: NEW_WATERSHED_GUIDE · DCC_SETUP

Terrain, stream network, and initial state inputs for DHSVM, from a watershed boundary and a DEM.

```
>>> [1] DEM entry (general: prep_dem)
    valid cells 22849 / 33231
>>> [4] hydrology (GRASS chain)
    22000.0 m2 / cell 100.00 m2 = 220 cells
[stream_network] segments=67  outlets=1
                 max_propagated_order=8
PIPELINE COMPLETE -- DHSVM input set
```

Recorded on dcc-login-05, August 16 (full session log in backup).

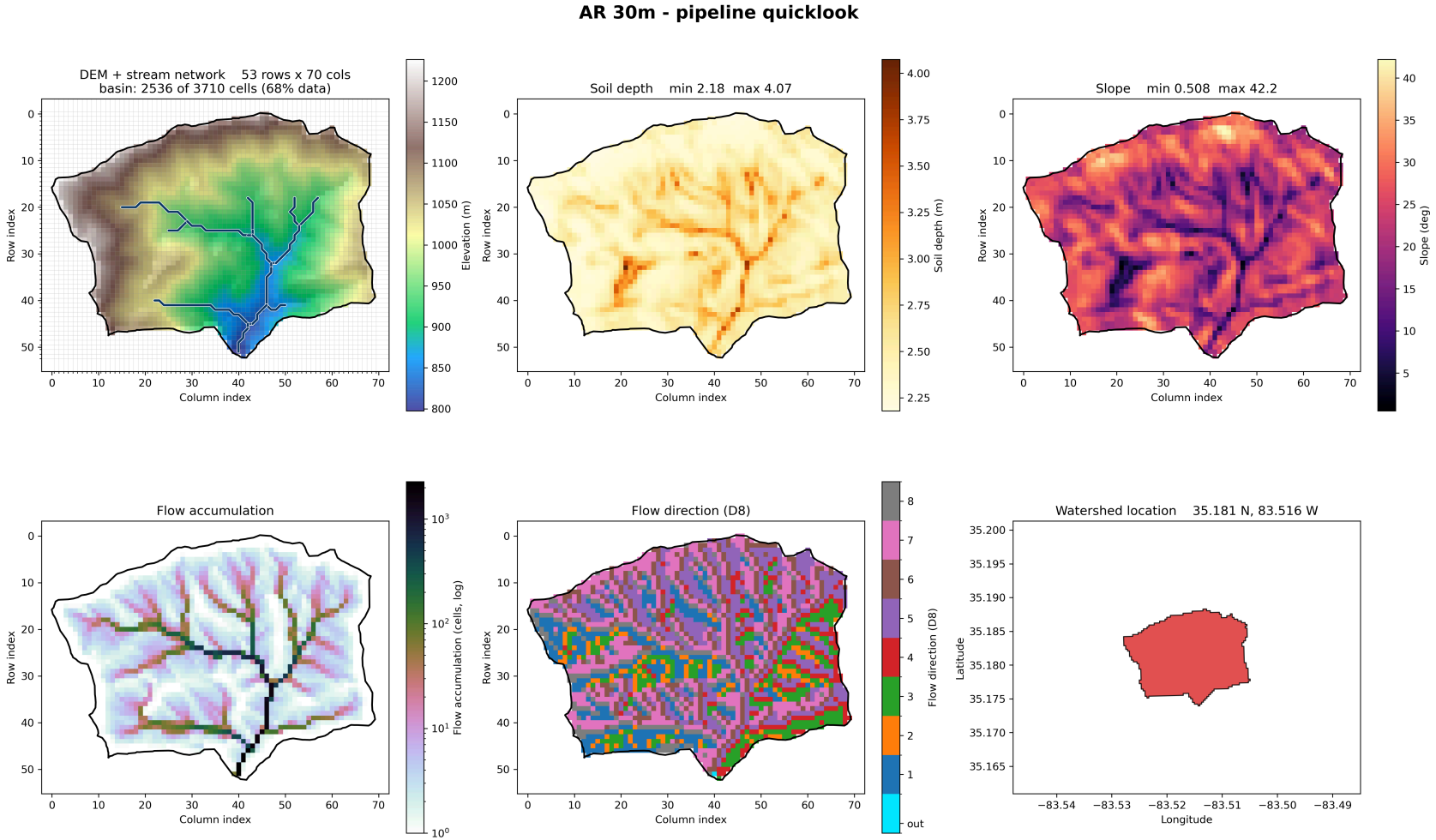
A real basin end to end: AR at 10 m



The pipeline's quick-look for AR (Arrowwood), built from its boundary shapefile at 10 m.

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks

The same watershed at 30 m



Only the resolution variable changed.

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks

Validation

Rebuilt against an audited reference implementation, with acceptance criteria fixed before porting: sha256 byte identity for the flat binaries, pixel-level equivalence for the rasters.

STAGE BY STAGE, AGAINST THE REFERENCE

clip	slope	base grids	hydrology	soil depth	stream network	model states
max |Δ| 0.0	max |Δ| 0.0	sha256 identical	max |Δ| 0.0	1 float32 ULP documented	2 junction ties documented	sha256 identical

END TO END

NSE 0.99999510
hourly streamflow, 2016 to 2018, standalone vs reference

PBIAS −0.000059%
same three years, same comparison

identical
skill scores against observed streamflow

The two amber stages are recorded in the validation log with cause and consequence: a math-library rounding difference of one bit, and one geometrically undefined tie resolved by the fixed rule.

Uses beyond DHSVM

Everything upstream of the DHSVM writers is general. Uses I can see directly:

Stream ecology and habitat. Network distance, fragmentation, and connectivity, computed from the directed network and its attribute table.

Geomorphology and erosion. Longitudinal profiles, knickpoints, and slope against drainage area from the same rasters.

Water quality and temperature. A defined downstream order for routing along the channel network.

Other hydrologic models. The same terrain and stream products; only a small writer for the target format is needed.

The repository is public under an MIT license and includes a step-by-step guide for a new watershed. I am glad to help anyone who wants to try it on their own basin.

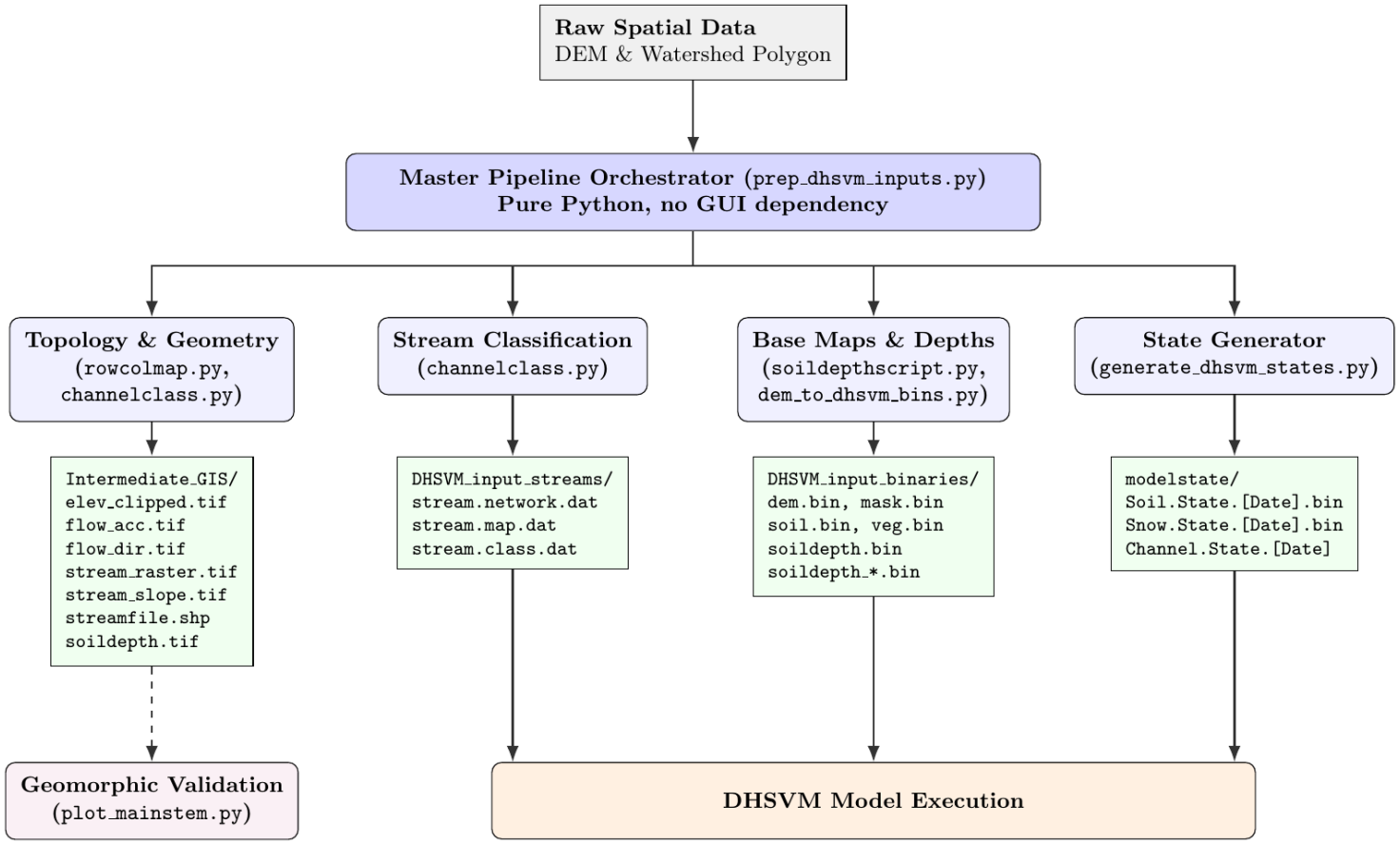
Summary and thanks

- **February**: scripts inside a **GUI**, on one machine, for one basin
- **Now**: a documented, **validated pipeline**, from a **boundary shapefile** on the **cluster**; two basins, three resolutions, end to end
- **From this fellowship**: **version control**, **validation** against a reference, **batch thinking**, and **floating-point care**

`github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks`

Thank you: Akshay and the CCT, Brad Murray and the lab, and my fellow fellows. I will answer questions in the shared document.

Backup: the consolidated orchestrator



An intermediate stage of the cleanup, drawn for my course report: one orchestrator, four modules, no GUI. The standalone pipeline in the repository descends from this design and removes the remaining QGIS dependency.

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks

Backup: the full DCC session log

```
rm -rf "$DHSVM_OUT"

(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ export DHSVM_DEM_SOURCE=/wo
ork/ys451/dhsvm_ca/standalone_dev/AR_src_3dep_10m.tif
(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ export DHSVM_DEM_SOURCE=/work/ys451/dhsvm_ca/standalone_dev/AR_src_3dep_10m.tif

(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ export DHSVM_STREAM_SOURCE_
_AREA_M2=22000
(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ export DHSVM_STREAM_SOURCE_AREA_M2=22000

(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ bash standalone_CA/pipeline
e/run_pipeline.sh
(dhsvm_geo) ys451@dcc-login-05  /work/ys451/cct_demo/DHSVM_Stream_Toolkit_ThatWorks $ bash standalone_CA/pipeline/run_pipeline.sh

============================================================
  STANDALONE DHSVM PREPROCESSING PIPELINE
  OUT = /work/ys451/cct_demo/outputs_AR_10m
============================================================

>>> [1] DEM entry (general: prep_dem)
    prep_dem /work/ys451/dhsvm_ca/standalone_dev/AR_src_3dep_10m.tif -> elev_clipped.tif  (res 10 m)
target: EPSG:32617  10.000 m  159 rows x 209 cols  (pad 0 cells)
extent (EPSG:32617): (269800.0, 3895230.0, 271890.0, 3896820.0)
source: EPSG:5070  351x309  res~(9.119545, 9.119545)  nodata=nan
wrote /work/ys451/cct_demo/outputs_AR_10m/elev_clipped.tif
  valid cells 22849 / 33231  (nodata 10382)
  elev range 791.2 .. 1229.9 m

>>> [2] slope (GRASS r.slope.aspect)
Default locale not found, using UTF-8
Default locale settings are missing. GRASS running with C locale.Starting GRASS GIS...
Creating new GRASS GIS location <gslope_908810>...
Executing <g.version> ...
GRASS 7.6.0 (2019)
Execution of <g.version> finished.
Cleaning up temporary files...
Default locale not found, using UTF-8
Default locale settings are missing. GRASS running with C locale.Starting GRASS GIS...
Executing <r.in.gdal input=/work/ys451/cct_demo/outputs_AR_10m/elev_clipped.tif output=elev_clipped --overwrite> ...
Importing raster map <elev_clipped>...  ... 100%
Execution of <r.in.gdal input=/work/ys451/cct_demo/outputs_AR_10m/elev_clipped.tif output=elev_clipped --overwrite> finished.
Cleaning up temporary files...
Default locale not found, using UTF-8
Default locale settings are missing. GRASS running with C locale.Starting GRASS GIS...
Executing <g.region raster=elev_clipped -p> ...
projection: 1 (UTM)
zone:       17
datum:      wgs84
ellipsoid:  wgs84
north:      3896820
south:      3895230
west:       269800
east:       271890
nsres:      10
ewres:      10
rows:       159
cols:       209
cells:      33231
Execution of <g.region raster=elev_clipped -p> finished.
Cleaning up temporary files...
Default locale not found, using UTF-8
Default locale settings are missing. GRASS running with C locale.Starting GRASS GIS...
Executing <r.slope.aspect elevation=elev_clipped slope=slope_raw format=degrees --overwrite> ...  ... 100%
```

Recorded with script on dcc-login-05, August 16. Scroll inside the box.

github.com/waveletswave/DHSVM_Stream_Toolkit_ThatWorks